Table of Contents

Objectives 3
1 Unsupervised Learning 3
1.1 Algorithms 3
1.2 Why Unsupervised Learning? 3
1.3 Types of unsupervised learning 3
2 K-Means Clustering 4
2.1 Using Built in Library 4
2.2 From scratch 9
3 K-Medoids Clustering 9
3.1 Using Built in Library 10
3.2 From scratch 13

1 Unsupervised Learning

Unsupervised Learning is a machine learning technique in which the users do not need to supervise the model. Instead, it allows the model to work on its own to discover patterns and information that was previously undetected. It mainly deals with the unlabelled data.

1.1 Algorithms

Unsupervised Learning Algorithms allow users to perform more complex processing tasks compared to supervised learning. Although, unsupervised learning can be more unpredictable compared with other natural learning methods. Unsupervised learning algorithms include clustering, anomaly detection, neural networks, etc

1.2 Why Unsupervised Learning?

Here, are prime reasons for using Unsupervised Learning:

- Unsupervised machine learning finds all kinds of unknown patterns in data.
- Unsupervised methods help you to find features which can be useful for categorization.
- It takes place in real time, so all the input data to be analyzed and labeled in the presence of learners.
- It is easier to get unlabeled data from a computer than labeled data, which needs manual intervention.

1.3 Types of unsupervised learning

Here, are prime reasons for using Unsupervised Learning:

1.3.1 Clustering

Clustering is an important concept when it comes to unsupervised learning. It mainly deals with finding a structure or pattern in a collection of uncategorized data. Clustering algorithms will process your data and find natural clusters(groups) if they exist in the data. You can also modify how many clusters your algorithms should identify. It allows you to adjust the granularity of these groups.

1.3.2 Types of Clustering

- Exclusive (partitioning): In this clustering method, Data is grouped in such a way that one data can belong to one cluster only. Example: K-means
- Agglomerative: In this clustering technique, every data is a cluster. The iterative unions between the two nearest clusters reduce the number of clusters. Example: Hierarchical clustering

2 K-Means Clustering

K means it is an iterative clustering algorithm which helps you to find the highest value for every iteration. Initially, the desired number of clusters are selected. In this clustering method, you need to cluster the data points into k groups. A larger k means smaller groups with more granularity in the same way. A lower k means larger groups with less granularity.

The output of the algorithm is a group of "labels." It assigns data points to one of the k groups. In k-means clustering, each group is defined by creating a centroid for each group. The centroids are like the heart of the cluster, which captures the points closest to them and adds them to the cluster.

The performance of K-Means is evaluated using a cost function, also known as the objective function or distortion function. It measures the total squared distance between each data point and the centroid of its assigned cluster. The algorithm aims to minimize this cost.

The K-Means cost function is defined as:

$$J = \sum_{n=1}^{N} \sum_{k=1}^{K} r_{nk} \|\mathbf{x}_n - \boldsymbol{\mu}_k\|^2$$

2.1 Using Built in Library

Make sure Python and pip are already installed on your system. To install Streamlit, run the following command in the command prompt or terminal:

2.1.1 Import Libraries

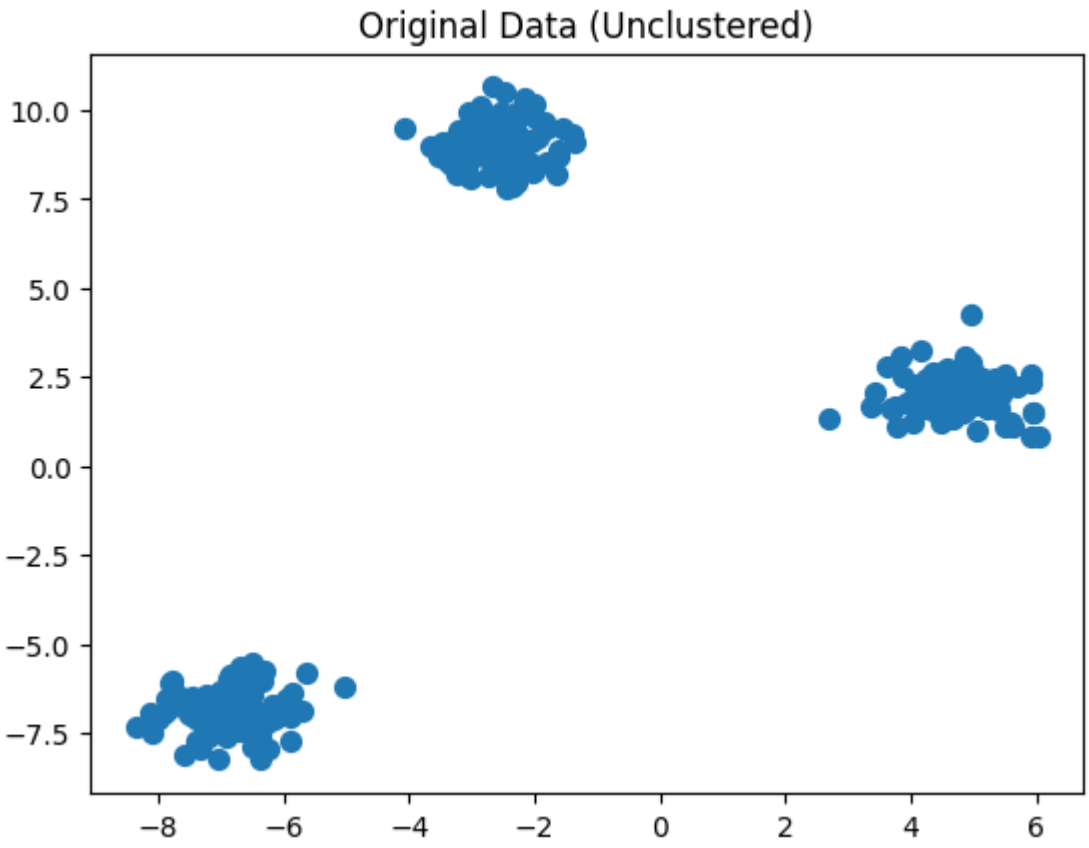
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs #to generate sample data
```

3.3.2 Create sample data

We'll use make_blobs to create synthetic 2D data points.

```
# Create dataset with 3 centers (clusters)
X, y_true = make_blobs(n_samples=300, centers=3, cluster_std=0.6,
random_state=42)
```

```
# Visualize the raw data
plt.scatter(X[:, 0], X[:, 1], s=50)
plt.title("Original Data (Unclustered)")
plt.show()
```



2.1.3 Apply K-Means clustering

```
# Create a KMeans instance with 3 clusters
kmeans = KMeans(n_clusters=3, random_state=42)

# Fit KMeans on the data
kmeans.fit(X)

# Get cluster centers and labels
centers = kmeans.cluster_centers_
labels = kmeans.labels_
```

2.1.4 Visualize the clustered data

```
# Plot clusters
plt.scatter(X[:, 0], X[:, 1], c=labels, s=50, cmap='viridis')

# Plot cluster centers
plt.scatter(centers[:, 0], centers[:, 1], c='red', s=200, alpha=0.7,
marker='X')
plt.title("K-Means Clustering Results")
plt.show()
```

2.1.5 Visualize the Elbow Method

Inertia measures how internally coherent the clusters are. It is essentially the same as the K-Means cost function, i.e., the sum of squared distances between each data point and the centroid of its assigned cluster.

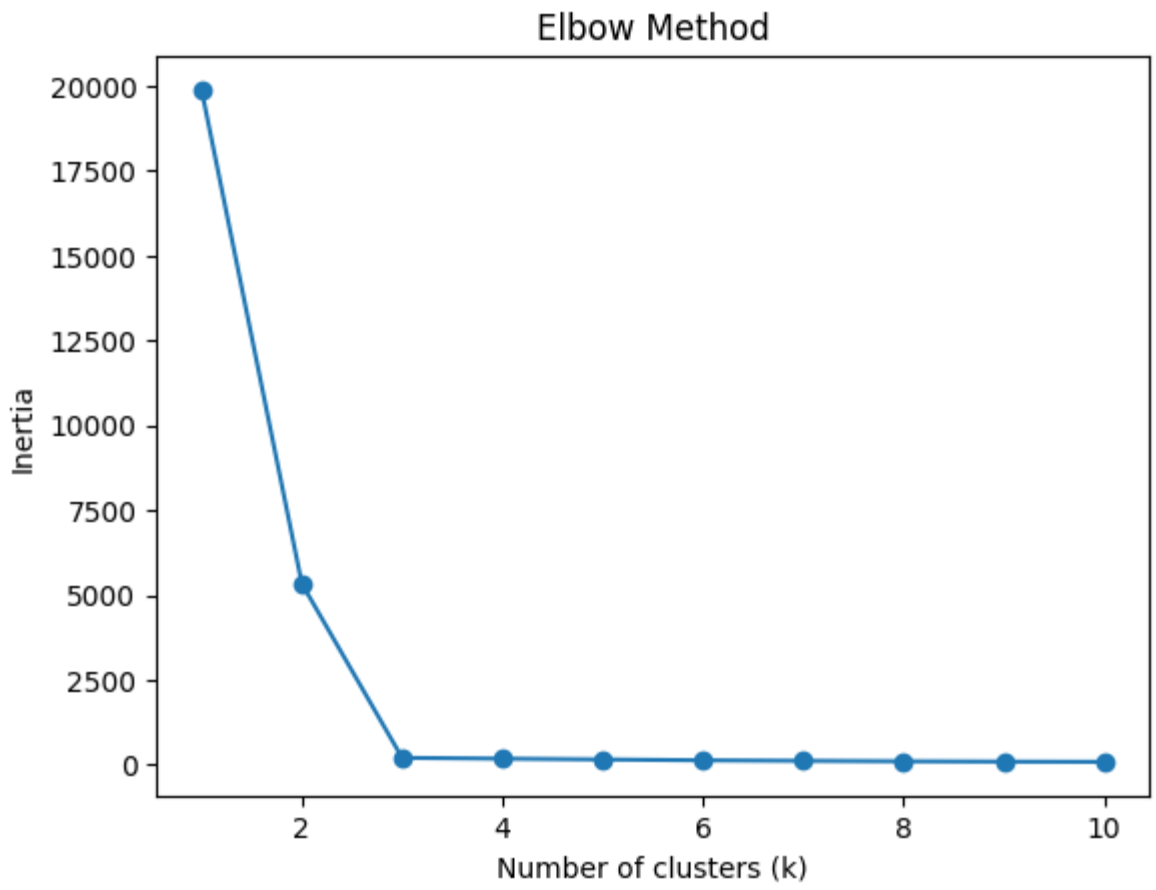
Formally, inertia corresponds to:

$$\text{Inertia} = \sum_{i=1}^{k} \sum_{x \in C_i} ||x - \mu_i||^2$$

Key points about inertia:

- Inertia is not a normalized metric, so it depends on the scale of your data.
- Lower inertia values indicate tighter clusters and better separation (but zero is only possible if every point is its own cluster).
- In high-dimensional spaces, Euclidean distances can become inflated, this is part of the curse of dimensionality.
- Running a dimensionality reduction technique such as PCA before applying K-Means can help mitigate this issue and speed up computation.

```
inertia = []  # Sum of squared distances to nearest cluster center

# Test k values from 1 to 10
for k in range(1, 11):
    km = KMeans(n_clusters=k, random_state=42)
    km.fit(X)
    inertia.append(km.inertia_)

# Plot the elbow curve
plt.plot(range(1, 11), inertia, marker='o')
plt.title("Elbow Method")
plt.xlabel("Number of clusters (k)")
plt.ylabel("Inertia")
plt.show()
```



2.2 From scratch

Partial from scratch implementation of k-means is provided here.

3 K-Medoids Clustering

K-Medoids is a clustering algorithm that groups data into k clusters, similar to K-Means. However, instead of using the mean (centroid) of points, it selects actual data points as cluster centers called medoids.

The most classical and widely used implementation of K-Medoids is the **PAM (Partitioning Around Medoids)** algorithm.

A medoid is the data point in a cluster whose total distance to all other points in the same cluster is minimum.

A larger k means smaller, more granular clusters and a smaller k mean larger clusters with less granularity

PAM works in two main phases:

1. **BUILD Phase (Initialization)**

- Select the first medoid randomly.
- Select the remaining medoids one by one.
- At each step, choose the point that reduces the total clustering cost the most.

This creates a strong initial set of medoids (better than purely random selection).

2. **SWAP Phase (Optimization)**
   - Assign each data point to the nearest medoid.
   - For each medoid m and non-medoid point o:
   - Swap m and o.
   - Compute new total cost.
   - If the swap reduces cost, keep it.
   - Repeat until no swap improves the solution.

   This guarantees local optimal clustering.

PAM minimizes the total dissimilarity between points and their assigned medoid.

$$\sum_{j=1}^{k} \sum_{i \in C_j} d(x_i, z_j)$$

Unlike K-Means, this cost function requires no squared distances and works with any distance metric (Euclidean, Manhattan, cosine, etc.). Lower the cost, better will be clustering.

3.1 Using Built in Library

You might need to `pip install scikit-learn-extra`

2.1.1 Import Libraries

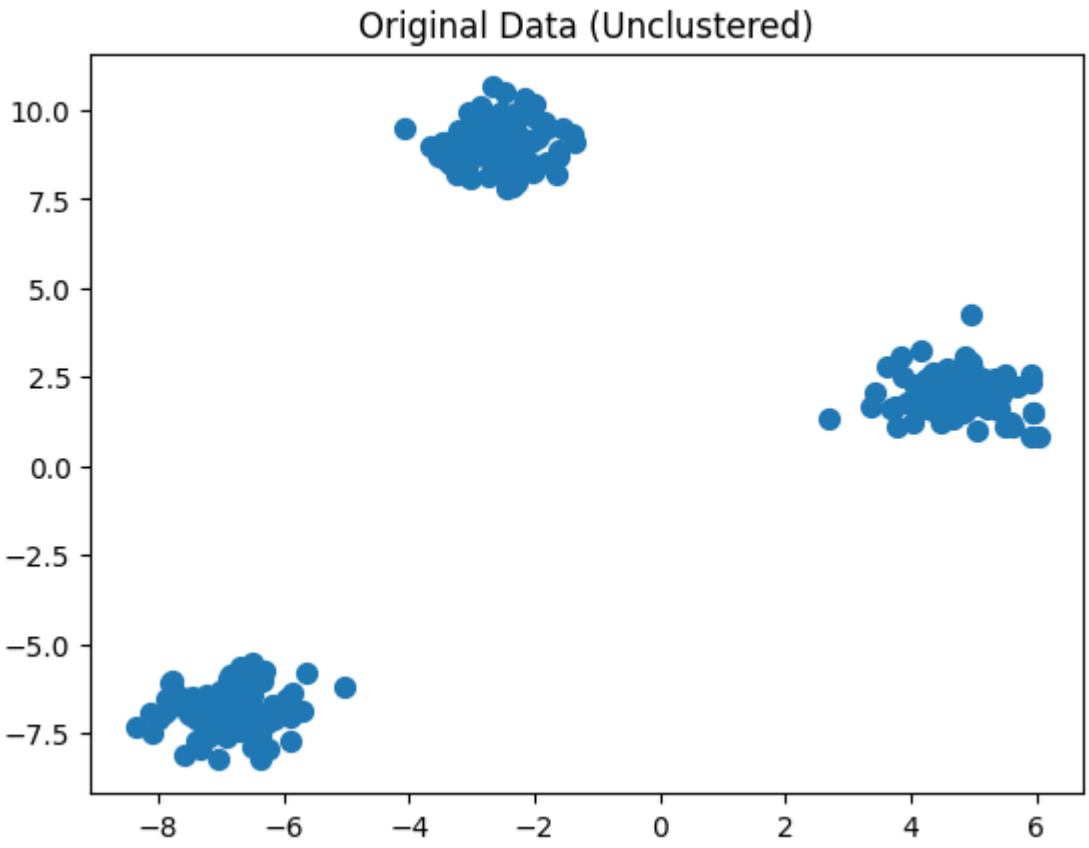
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn_extra.cluster import KMedoids
from sklearn.datasets import make_blobs
```

3.3.2 Create sample data

We'll use make_blobs to create synthetic 2D data points.

```
X, y_true = make_blobs(n_samples=300, centers=3, cluster_std=0.6,
random_state=42)
```

```
plt.scatter(X[:, 0], X[:, 1], s=50)
plt.title("Original Data (Unclustered)")
plt.show()
```

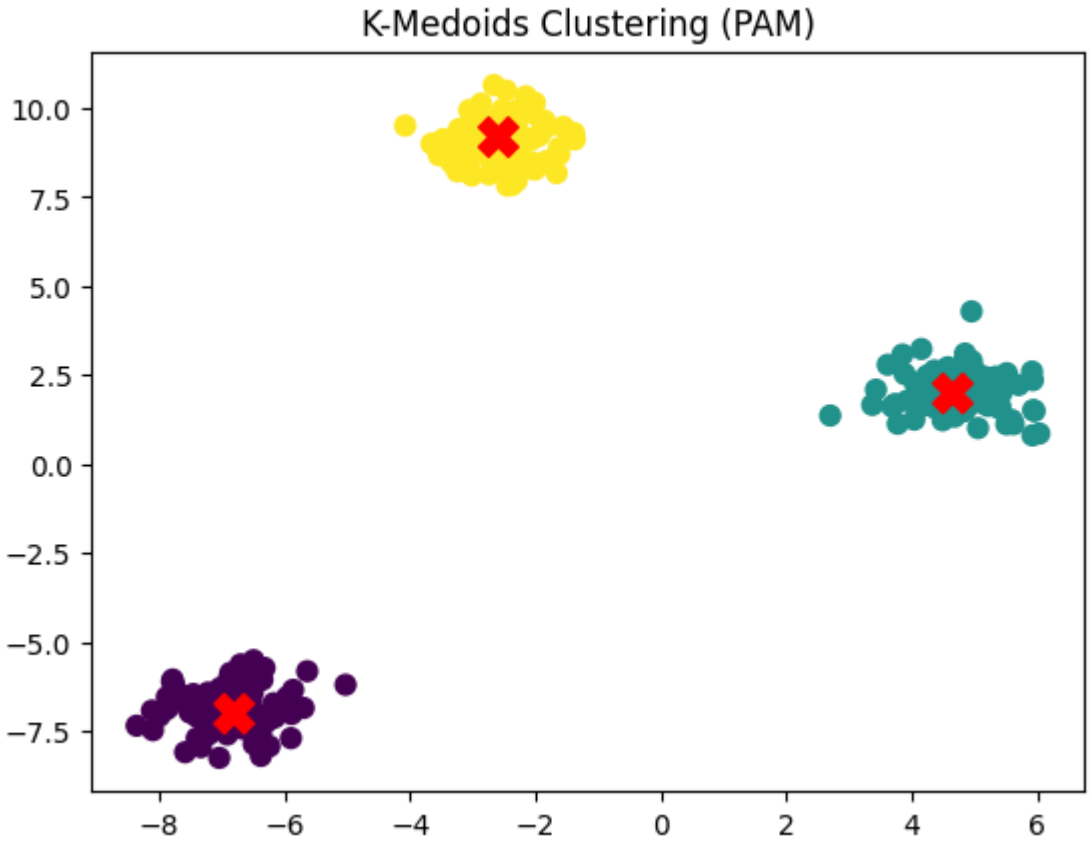


2.1.3 Apply K-Means clustering

```
kmedoids = KMedoids(n_clusters=3,method='pam',random_state=42)

kmedoids.fit(X)

medoids = kmedoids.cluster_centers_
labels = kmedoids.labels_
```

2.1.4 Visualize the clustered data

```
plt.scatter(X[:, 0], X[:, 1], c=labels, s=50, cmap='viridis')
plt.scatter(medoids[:, 0], medoids[:, 1], c='red', s=200, marker='X')

plt.title("K-Medoids Clustering (PAM)")
plt.show()
```

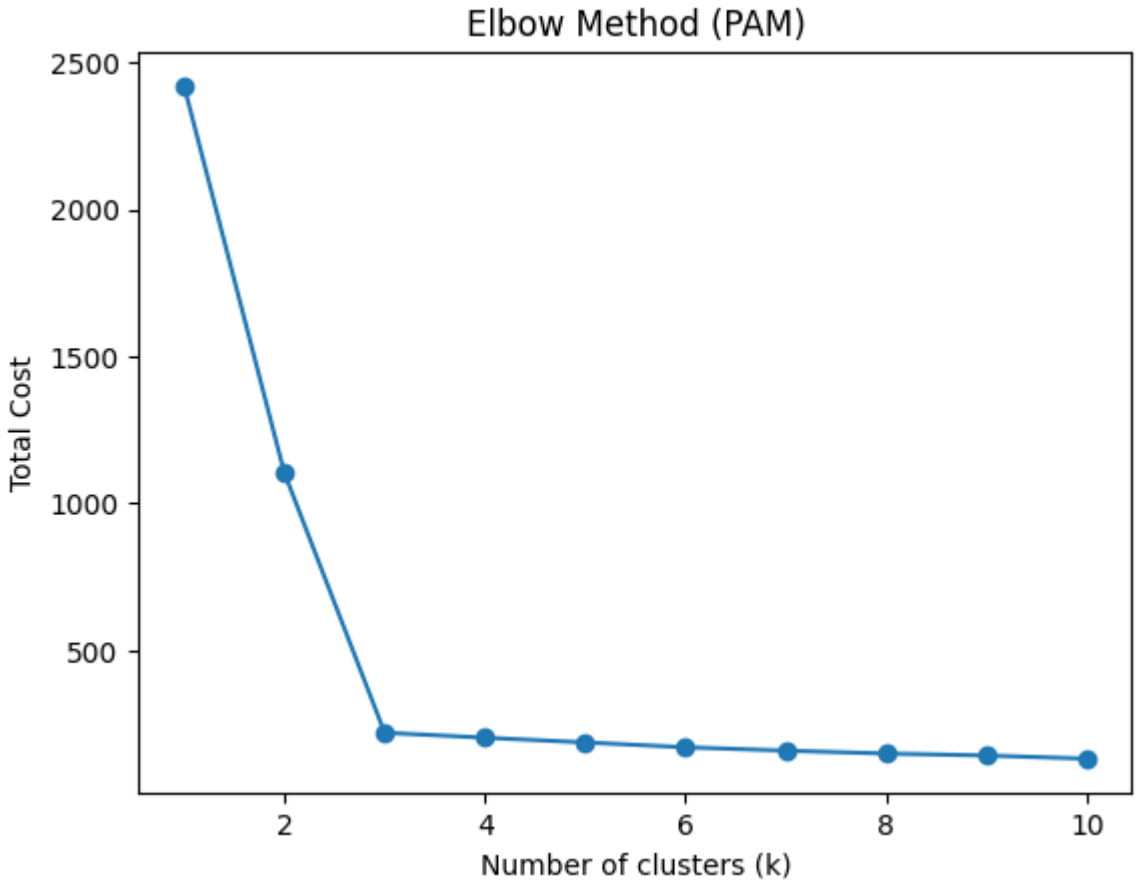


2.1.5 Visualize the Elbow Method

Instead of inertia (squared distance), PAM uses total distance.

```
cost = []

for k in range(1, 11):
    model = KMedoids(n_clusters=k,
                     method='pam',
                     random_state=42)
    model.fit(X)
```

```
    cost.append(model.inertia_)

plt.plot(range(1, 11), cost, marker='o')
plt.title("Elbow Method (PAM)")
plt.xlabel("Number of clusters (k)")
plt.ylabel("Total Cost")
plt.show()
```



3.2 From scratch

Explore the from scratch implementation of k-medoids.